NUS National University of Singapore | Advanced Computing for Executives School of Computing

Reading 6.4: Tabular Q-learning

Welcome to this lesson on Tabular Q-Learning, a foundational TD control algorithm introduced in 1989. By the end of this video, you will:

- Understand the difference between on-policy and off-policy learning
- Uncover the update rule for Q-learning
- Apply Q-learning on a simple Gridworld example

Let's dive into the world of off-policy learning and see how Q-learning helps agents make optimal decisions!

Before embarking on Q-learning we need to clarify the difference between on-policy and off-policy learning. Up to now we have only discussed on-policy learning. Let's call the policy that the agent is learning the target policy $\pi(s|a)$. An example of a target policy is the optimal policy π_*. In on-policy learning we evaluate and improve the target policy, the same policy that is used to select actions.

Let's call the policy that the agent uses to select actions the behaviour policy $b(s|a)$. In off-policy learning the behaviour policy is different from the target policy. This allows the agent to explore more states using an exploratory behaviour policy while learning the optimal target policy.

Q-learning is an example of off-policy learning, where the agent learns optimal actions independently of its exploration policy. Q-learning converges to a deterministic optimal policy.

The rule updates the Q-value by combining the current estimate with new information derived from the observed reward and the estimated best future rewards. $max_{a'}$ refers to the maximum Q-value from the next state

$$Q_\pi(s_t, a_t) \leftarrow Q_\pi(s_t, a_t) + \alpha[r_{t+1} + \gamma max_{a'} Q_\pi(s_{t+1}, a_{t+1}) - Q_\pi(s_t, a_t)]$$

Try it Yourself:

Now that you have seen the Q-learning update rule, apply what you learned on a simple Gridworld example with 6 states. The agent starts on tile 2 and ends on tile 6. The agent receives 10 points for reaching tile 6 and -1 points for all other tiles. From each tile, the agent can choose to go (up, down, left, right), however, the agent cannot move outside the given tiles.

The initial Q(s,a) values are provided and N/A means that the action is not possible in that state.

State	Up	Down	Left	Right
1	N/A	4	N/A	6
2	N/A	3	2	4
3	N/A	7	5	N/A
4	8	N/A	N/A	4
5	8	N/A	5	6
6	N/A	N/A	N/A	N/A

Task: with learning rate = 0.25 and discount rate = 0.75, if the agent travels down first then right, what are the updated values in the Q-table.

1. **Initial Q-value for "down" from tile 2**:

$$Q(2, \text{down}) = 3$$

2. **Reward for moving down**:

$$r = -1 \text{ (as moving to tile 5 gives -1 point)}$$

3. **Maximum Q-value for the next state (tile 5)**:

$$\max_a Q(5, a) = Q(5, \text{right}) = 6$$

4. **Apply Q-learning update rule**:

$$Q(2, \text{down}) \leftarrow 3 + 0.25 \left[-1 + 0.75 \times 6 - 3\right]$$

Breaking it down:

- $-1 + 0.75 \times 6 = -1 + 4.5 = 3.5$
- $3.5 - 3 = 0.5$
- $0.25 \times 0.5 = 0.125$
- $3 + 0.125 = 3.125$

So, the **updated Q-value** for moving down from tile 2 is:

$$Q(2, \text{down}) = 3.125$$

1

1. **Initial Q-value for "right" from tile 5**:

$$Q(5, \text{right}) = 6$$

2. **Reward for moving right**:

$$r = 10 \text{ (as moving to tile 6 gives 10 points)}$$

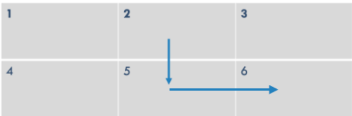
3. **There is no next state** since tile 6 is the goal, so we don't apply the discount factor to future states.

4. **Apply Q-learning update rule**:

$$Q(5, \text{right}) \leftarrow 6 + 0.25\,[10 - 6]$$

Breaking it down:

- $10 - 6 = 4$
- $0.25 \times 4 = 1$
- $6 + 1 = 7$



So, the **updated Q-value** for moving right from tile 5 is:

$$(\ , \text{right}) = 7$$

State	Up	Down	Left	Right
1	N/A	4	N/A	6
2	N/A	~~3~~ 3.12	2	4
3	N/A	7	5	N/A
4	8	N/A	N/A	4
5	8	N/A	5	~~6~~ 7
6	N/A	N/A	N/A	N/A